



MEMBERS

Name	SAP ID
Ayush Negi	590015410
Ayush Kumar	590014539
Aditi Vijay	590018228
Mayank Tanishk	590015863
Mayank Parmar	590017039
Sakshi Singh	590012402
Samiksha Rajan	590016234
Satyam Pandey	590016569



INDEX

Section	Content Description	Page No.
1. Executive Summary and Phase Goal	Overview of building the main Swing screens and connecting them to the service and DAO layers so the application works end to end.	3
2. Architectural Adherence and System Flow	Detailed explanation of the system flow (UI → Service → DAO → DB) and strict adherence to rules like avoiding direct SQL calls from UI event handlers.	3 - 4
3. Detailed UI Screen Implementations	Descriptions of the interactive Swing screens built: Dashboard, Company Management, Drive Management, Student Application, Shortlisting, and Interview Schedule.	4 - 21
4. Integration Strategy and Execution Order	The step-by-step approach taken: building navigation, connecting form actions to services, services to DAO, and showing clear UI success/error messages.	21 - 22
5. Input Validation Strategy	Details on how the project visibly validates data before committing and properly blocks invalid inputs or empty forms.	22 - 24
6. Phase 4 Milestone Review Checklist	Verification that one complete workflow functions correctly, actions are connected to service logic, and the project is usable.	24

Object Oriented Programming (CSEG1044)

Phase 4: UI and Integration

Project Title: Placement Drive Management System.

- **Goal of the Phase**

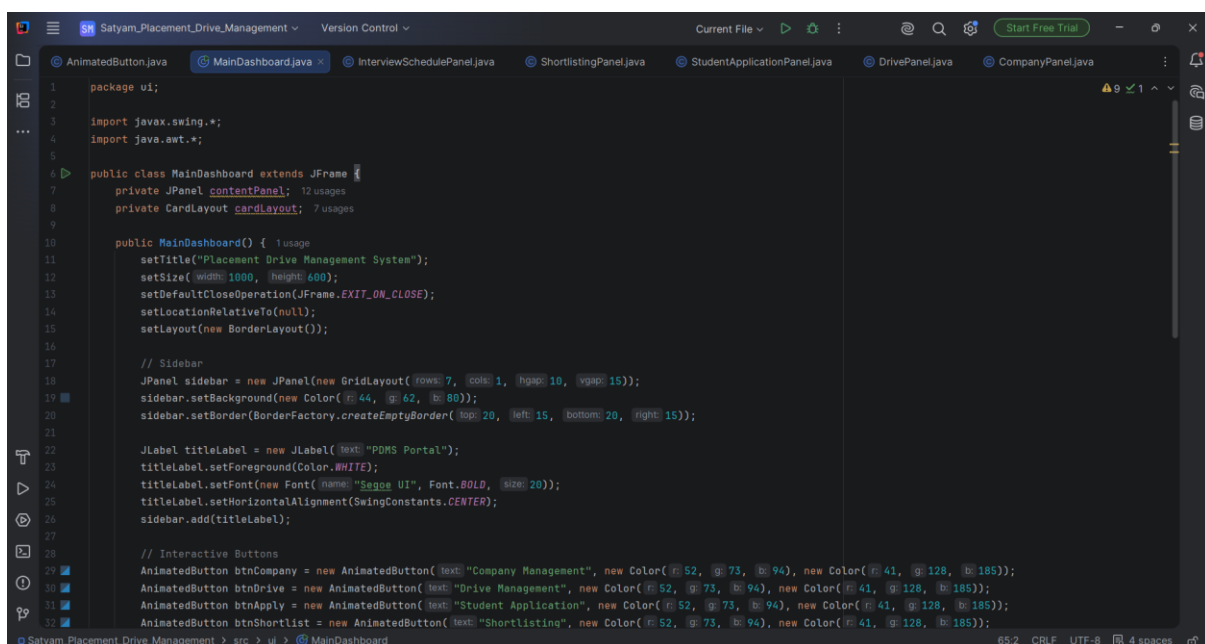
- The primary goal of Phase 4 is to build the main Java Swing graphical user interface (GUI) screens and seamlessly connect them to the service and Data Access Object (DAO) layers. This ensures that the application functions end-to-end, translating user interactions into secure database transactions while adhering to strict Object-Oriented design principles.

- **Architectural Adherence and System Flow**

- **This phase strictly follows the clean architecture established in Phase 3. The system flow dictates that interactions move sequentially: User → UI Dashboard → Service Layer (Logic Handler) → DAO Layer → DBConnection → MySQL Database.**
- **Separation of Concerns:** A core design rule strictly enforced in this phase is that absolutely no SQL queries are written within the UI layer.
- **Event Handling:** All UI button actions (e.g., actionPerformed events) are restricted to capturing

input, performing visible UI validation, and delegating the data to the appropriate service methods.

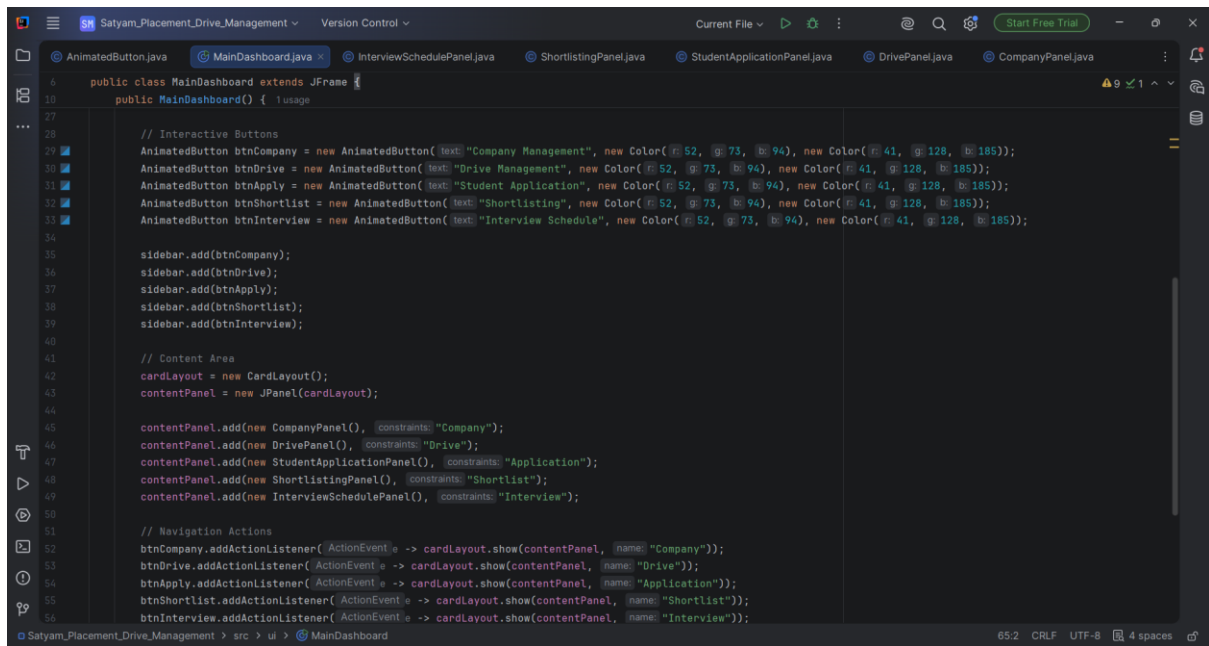
- **Detailed UI Screen Implementations**
- **To facilitate the operational flow of the system , the following interactive Swing screens were developed. To enhance user interaction beyond standard Swing components, a custom AnimatedButton utility utilizing javax.swing.Timer was implemented to provide smooth color-fade animations on mouse hover.**
- **Main Dashboard (MainDashboard.java):**
- Acts as the primary container utilizing a CardLayout to manage sub-panels, preventing the UI anti-pattern of opening multiple overlapping windows.
- Features a persistent sidebar for clear, simple navigation between all major application modules.



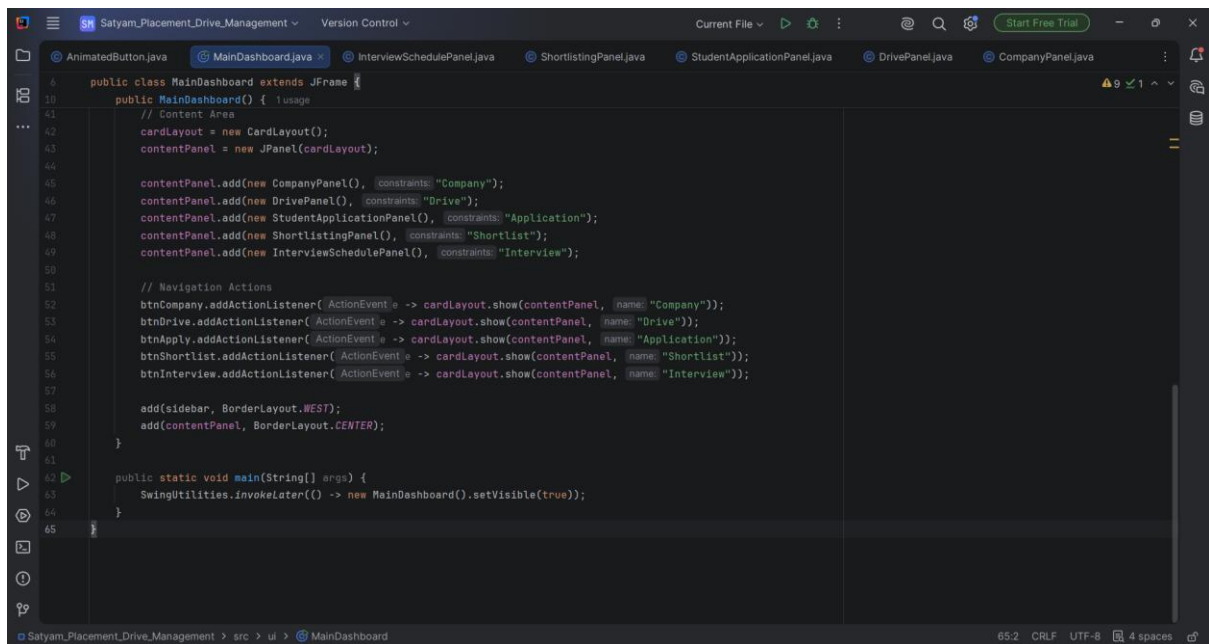
```

1 package ui;
2
3 import javax.swing.*;
4 import java.awt.*;
5
6 public class MainDashboard extends JFrame {
7     private JPanel contentPanel;
8     private CardLayout cardLayout;
9
10    public MainDashboard() {
11        setTitle("Placement Drive Management System");
12        setSize(1000, 600);
13        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
14        setLocationRelativeTo(null);
15        setLayout(new BorderLayout());
16
17        // Sidebar
18        JPanel sidebar = new JPanel(new GridLayout(7, 1, 10, 15));
19        sidebar.setBackground(new Color(44, 62, 80));
20        sidebar.setBorder(BorderFactory.createEmptyBorder(20, 15, 20, 15));
21
22        JLabel titleLabel = new JLabel("PDMS Portal");
23        titleLabel.setForeground(Color.WHITE);
24        titleLabel.setFont(new Font("Segoe UI", Font.BOLD, 20));
25        titleLabel.setHorizontalAlignment(SwingConstants.CENTER);
26        sidebar.add(titleLabel);
27
28        // Interactive Buttons
29        AnimatedButton btnCompany = new AnimatedButton("Company Management", new Color(52, 73, 94), new Color(41, 128, 185));
30        AnimatedButton btnDrive = new AnimatedButton("Drive Management", new Color(52, 73, 94), new Color(41, 128, 185));
31        AnimatedButton btnApply = new AnimatedButton("Student Application", new Color(52, 73, 94), new Color(41, 128, 185));
32        AnimatedButton btnShortlist = new AnimatedButton("Shortlisting", new Color(52, 73, 94), new Color(41, 128, 185));
    }
}

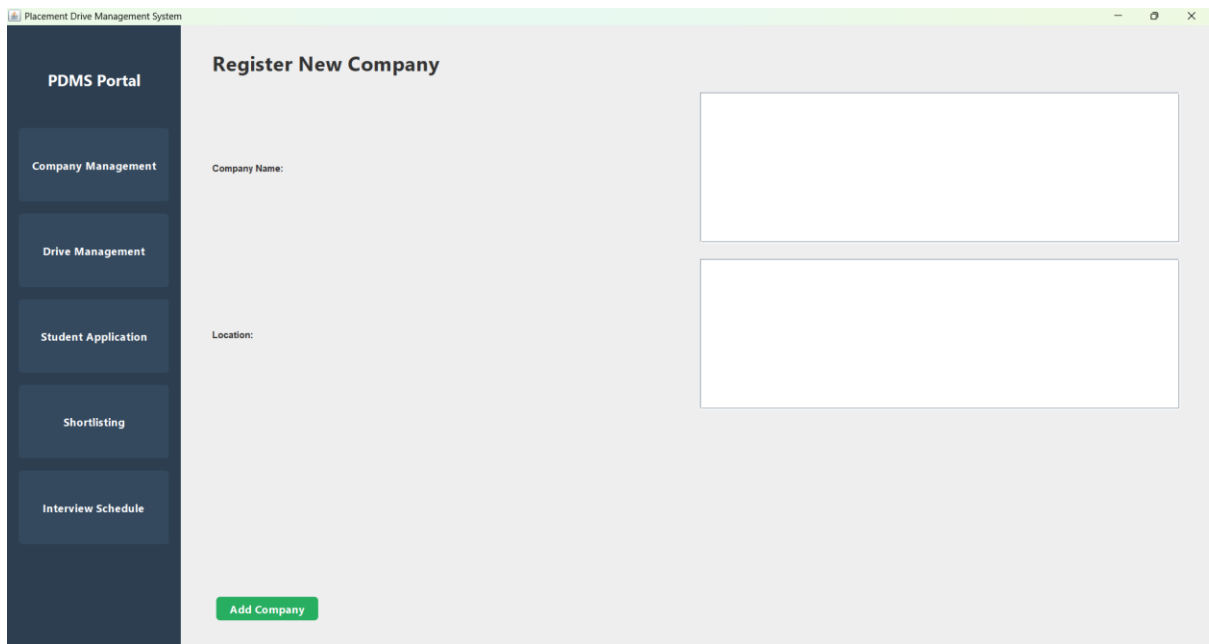
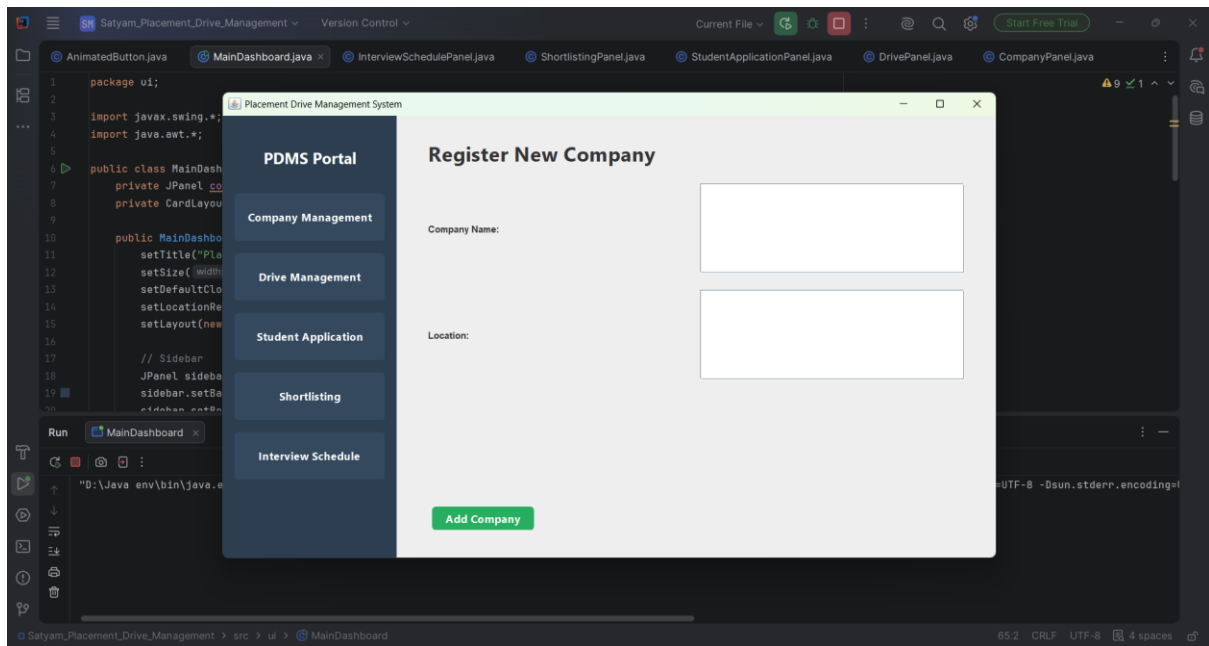
```



```
6 public class MainDashboard extends JFrame {
10     public MainDashboard() {
11         // Interactive Buttons
12         AnimatedButton btnCompany = new AnimatedButton(text: "Company Management", new Color(52, 73, 94), new Color(41, 128, 185));
13         AnimatedButton btnDrive = new AnimatedButton(text: "Drive Management", new Color(52, 73, 94), new Color(41, 128, 185));
14         AnimatedButton btnApply = new AnimatedButton(text: "Student Application", new Color(52, 73, 94), new Color(41, 128, 185));
15         AnimatedButton btnShortlist = new AnimatedButton(text: "Shortlisting", new Color(52, 73, 94), new Color(41, 128, 185));
16         AnimatedButton btnInterview = new AnimatedButton(text: "Interview Schedule", new Color(52, 73, 94), new Color(41, 128, 185));
17
18         sidebar.add(btnCompany);
19         sidebar.add(btnDrive);
20         sidebar.add(btnApply);
21         sidebar.add(btnShortlist);
22         sidebar.add(btnInterview);
23
24         // Content Area
25         cardLayout = new CardLayout();
26         contentPanel = new JPanel(cardLayout);
27
28         contentPanel.add(new CompanyPanel(), constraints: "Company");
29         contentPanel.add(new DrivePanel(), constraints: "Drive");
30         contentPanel.add(new StudentApplicationPanel(), constraints: "Application");
31         contentPanel.add(new ShortlistingPanel(), constraints: "Shortlist");
32         contentPanel.add(new InterviewSchedulePanel(), constraints: "Interview");
33
34         // Navigation Actions
35         btnCompany.addActionListener(new ActionListener() {
36             public void actionPerformed(ActionEvent e) {
37                 cardLayout.show(contentPanel, "Company");
38             }
39         });
40         btnDrive.addActionListener(new ActionListener() {
41             public void actionPerformed(ActionEvent e) {
42                 cardLayout.show(contentPanel, "Drive");
43             }
44         });
45         btnApply.addActionListener(new ActionListener() {
46             public void actionPerformed(ActionEvent e) {
47                 cardLayout.show(contentPanel, "Application");
48             }
49         });
50         btnShortlist.addActionListener(new ActionListener() {
51             public void actionPerformed(ActionEvent e) {
52                 cardLayout.show(contentPanel, "Shortlist");
53             }
54         });
55         btnInterview.addActionListener(new ActionListener() {
56             public void actionPerformed(ActionEvent e) {
57                 cardLayout.show(contentPanel, "Interview");
58             }
59         });
60     }
61 }
```

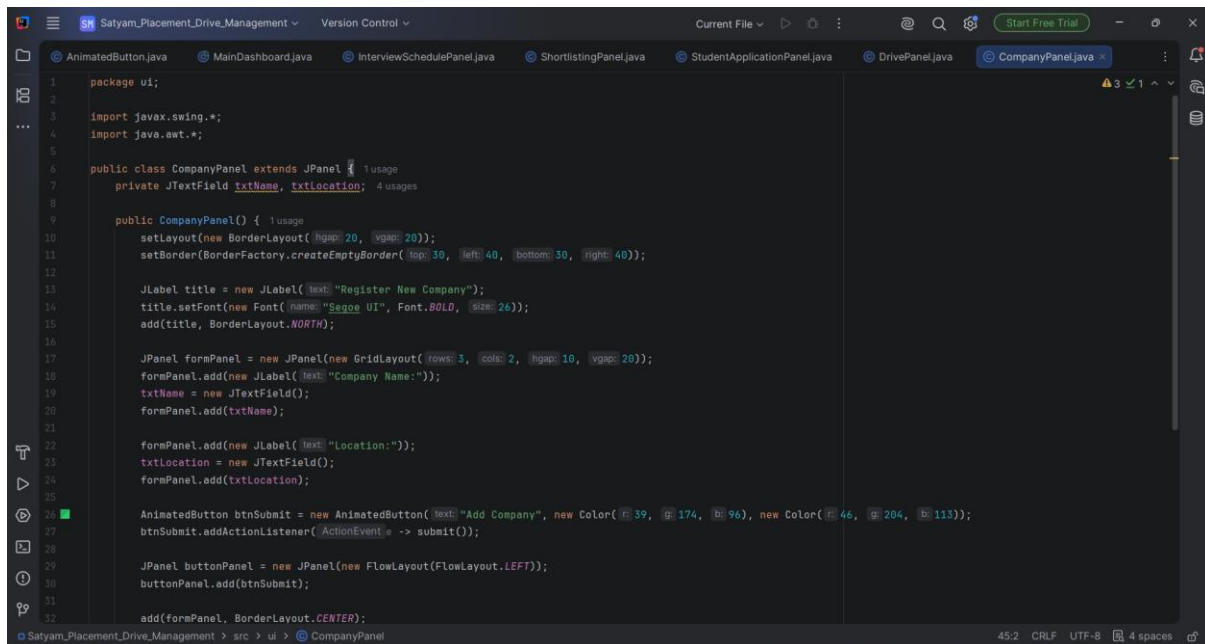


```
62
63
64 // Navigation Actions
65 btnCompany.addActionListener(new ActionListener() {
66     public void actionPerformed(ActionEvent e) {
67         cardLayout.show(contentPanel, "Company");
68     }
69 });
70 btnDrive.addActionListener(new ActionListener() {
71     public void actionPerformed(ActionEvent e) {
72         cardLayout.show(contentPanel, "Drive");
73     }
74 });
75 btnApply.addActionListener(new ActionListener() {
76     public void actionPerformed(ActionEvent e) {
77         cardLayout.show(contentPanel, "Application");
78     }
79 });
80 btnShortlist.addActionListener(new ActionListener() {
81     public void actionPerformed(ActionEvent e) {
82         cardLayout.show(contentPanel, "Shortlist");
83     }
84 });
85 btnInterview.addActionListener(new ActionListener() {
86     public void actionPerformed(ActionEvent e) {
87         cardLayout.show(contentPanel, "Interview");
88     }
89 });
90
91 add(sidebar, BorderLayout.WEST);
92 add(contentPanel, BorderLayout.CENTER);
93
94 }
95
96 public static void main(String[] args) {
97     SwingUtilities.invokeLater(() -> new MainDashboard().setVisible(true));
98 }
99 }
```

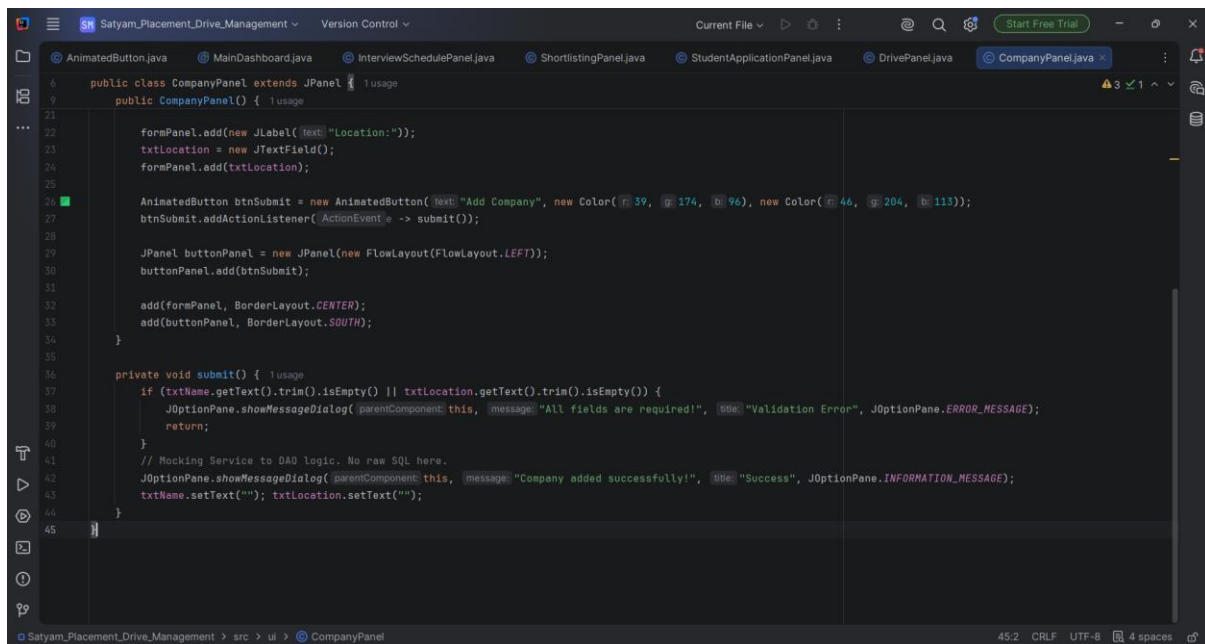


- **Company Management Screen (CompanyPanel.java):**
- **Purpose:** Facilitates Step 1 of the operational flow: "Add Company".
- **Fields:** Captures Company Name and Location.

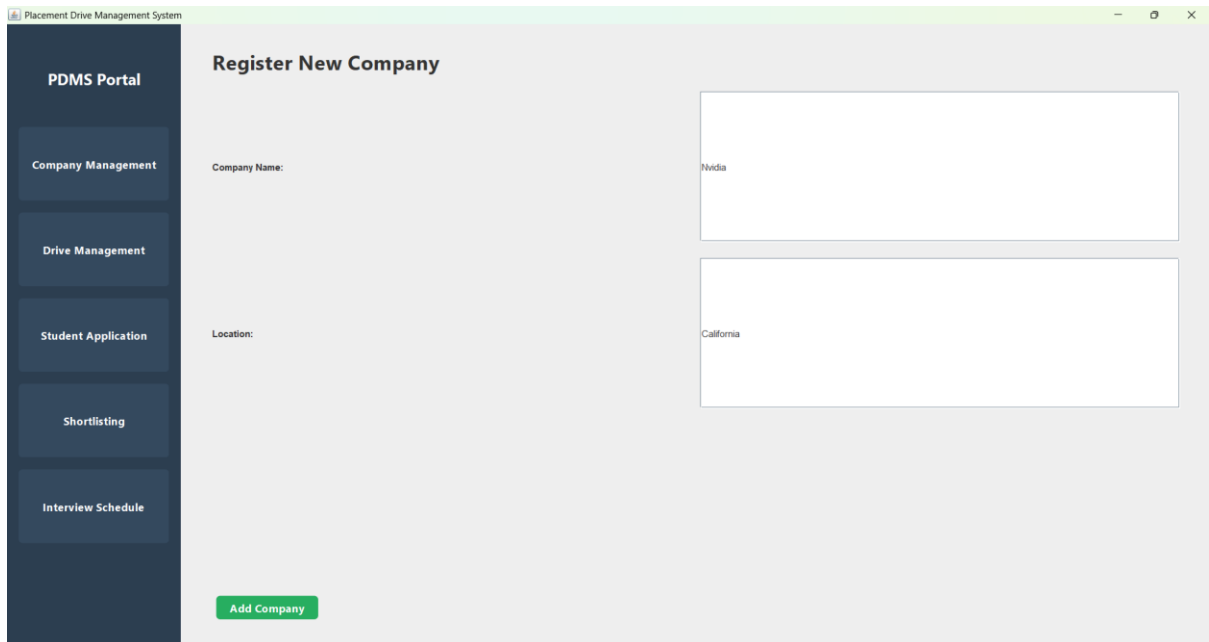
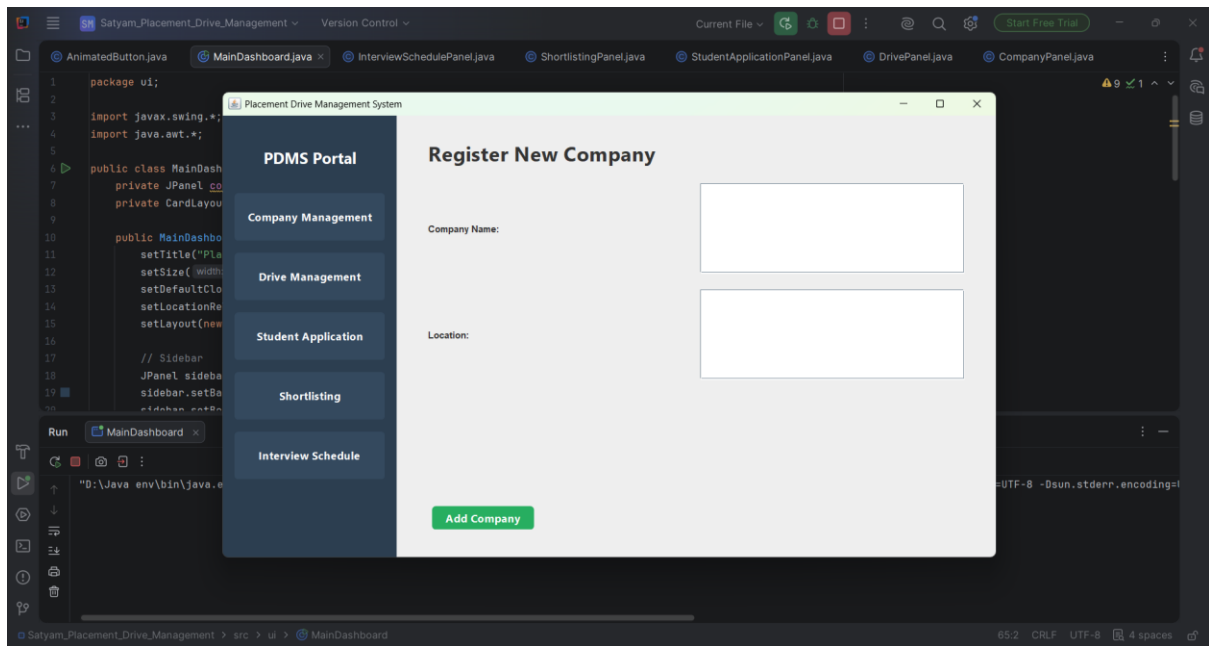
- **Action:** Triggers the addition of a company record, ultimately passing data to `CompanyDAO.addCompany()`.

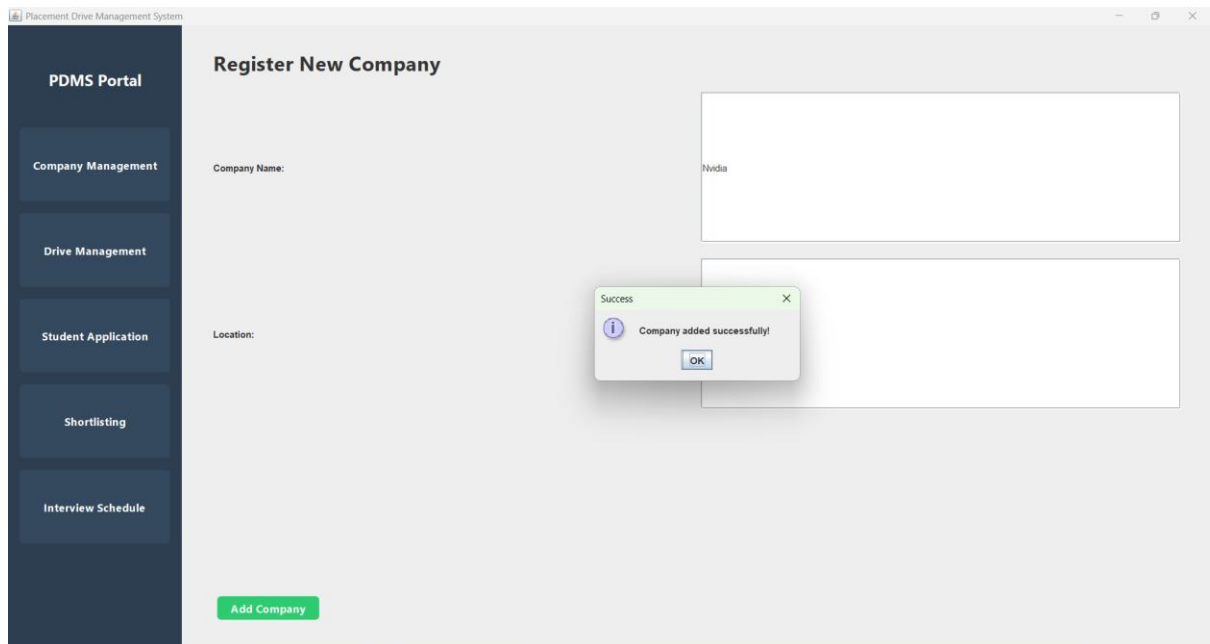


```
1 package ui;
2
3 import javax.swing.*;
4 import java.awt.*;
5
6 public class CompanyPanel extends JPanel { 1 usage
7     private JTextField txtName, txtLocation; 4 usages
8
9     public CompanyPanel() { 1 usage
10         setLayout(new BorderLayout(10, 10));
11         setBorder(BorderFactory.createEmptyBorder(10, 10, 10, 10));
12
13         JLabel title = new JLabel("Register New Company");
14         title.setFont(new Font("Segoe UI", Font.BOLD, 26));
15         add(title, BorderLayout.NORTH);
16
17         JPanel formPanel = new JPanel(new GridLayout(2, 1, 10, 10));
18         formPanel.add(new JLabel("Company Name:"));
19         txtName = new JTextField();
20         formPanel.add(txtName);
21
22         formPanel.add(new JLabel("Location:"));
23         txtLocation = new JTextField();
24         formPanel.add(txtLocation);
25
26         AnimatedButton btnSubmit = new AnimatedButton("Add Company", new Color(39, 174, 96), new Color(46, 204, 113));
27         btnSubmit.addActionListener(e -> submit());
28
29         JPanel buttonPanel = new JPanel(new FlowLayout(FlowLayout.LEFT));
30         buttonPanel.add(btnSubmit);
31
32         add(formPanel, BorderLayout.CENTER);
33     }
34 }
```

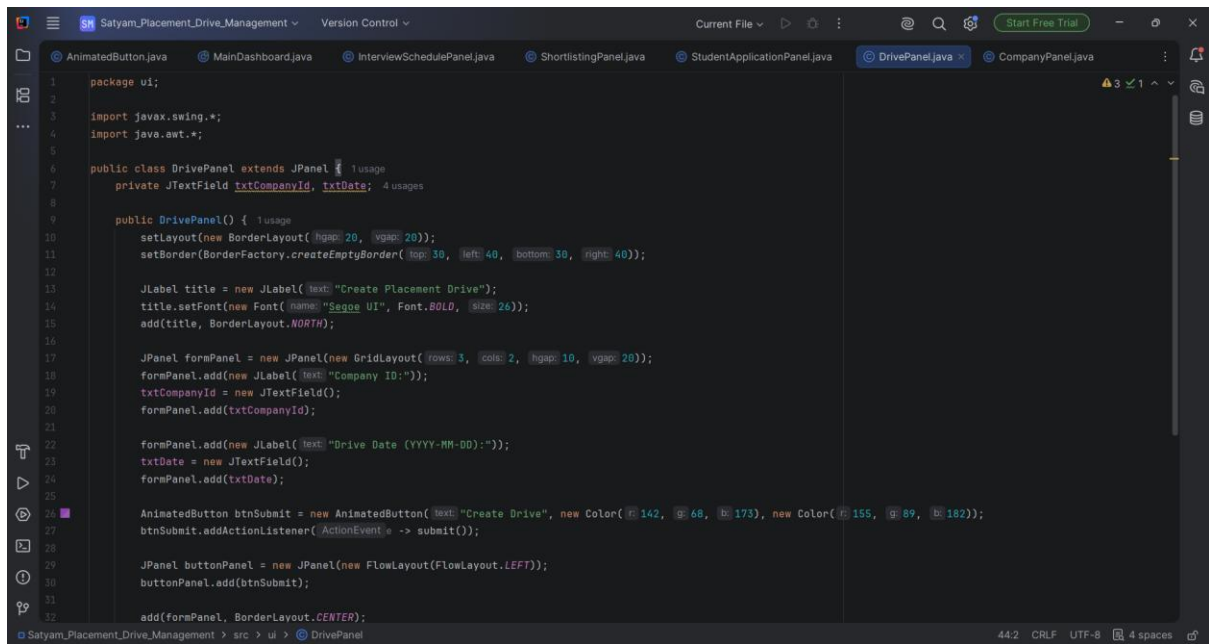


```
21
22     formPanel.add(new JLabel("Location:"));
23     txtLocation = new JTextField();
24     formPanel.add(txtLocation);
25
26     AnimatedButton btnSubmit = new AnimatedButton("Add Company", new Color(39, 174, 96), new Color(46, 204, 113));
27     btnSubmit.addActionListener(e -> submit());
28
29     JPanel buttonPanel = new JPanel(new FlowLayout(FlowLayout.LEFT));
30     buttonPanel.add(btnSubmit);
31
32     add(formPanel, BorderLayout.CENTER);
33     add(buttonPanel, BorderLayout.SOUTH);
34 }
35
36 private void submit() { 1 usage
37     if (txtName.getText().trim().isEmpty() || txtLocation.getText().trim().isEmpty()) {
38         JOptionPane.showMessageDialog(parentComponent, this, "All fields are required!", "Validation Error", JOptionPane.ERROR_MESSAGE);
39         return;
40     }
41     // Mocking Service to DAO logic. No raw SQL here.
42     JOptionPane.showMessageDialog(parentComponent, this, "Company added successfully!", "Success", JOptionPane.INFORMATION_MESSAGE);
43     txtName.setText(""); txtLocation.setText("");
44 }
45 }
```

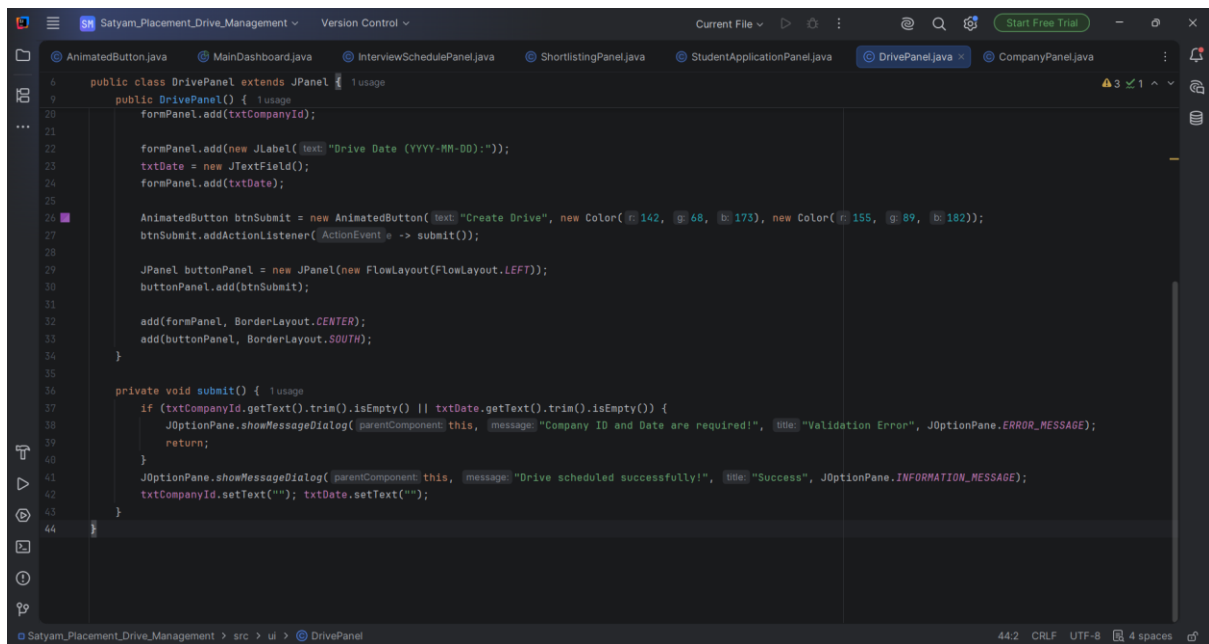




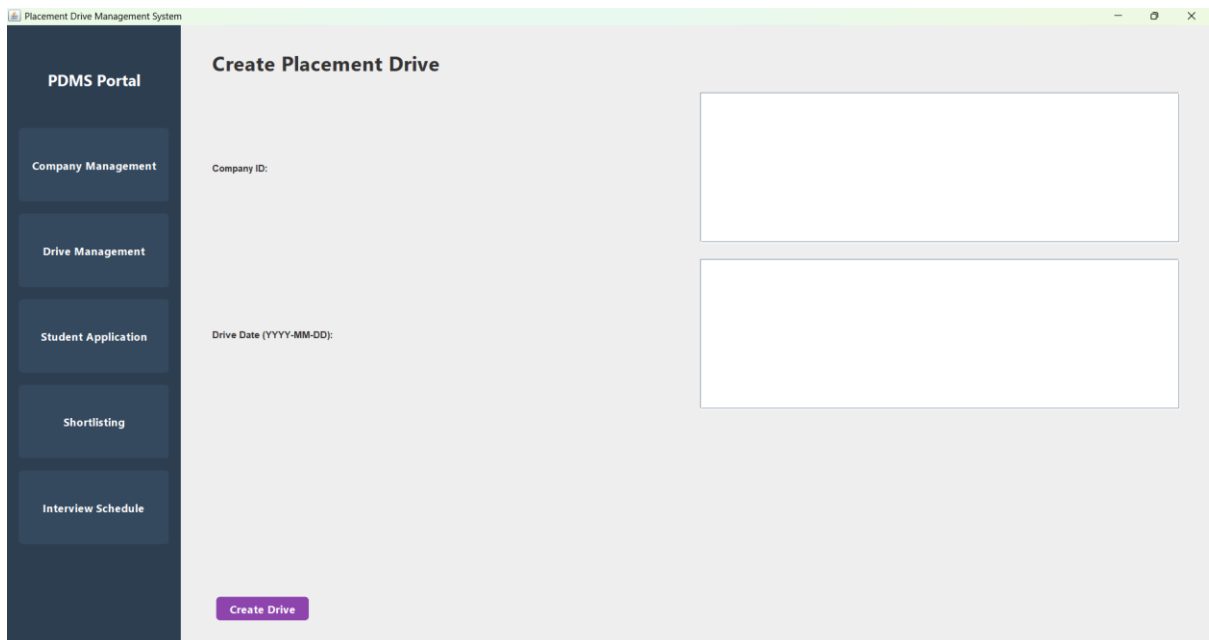
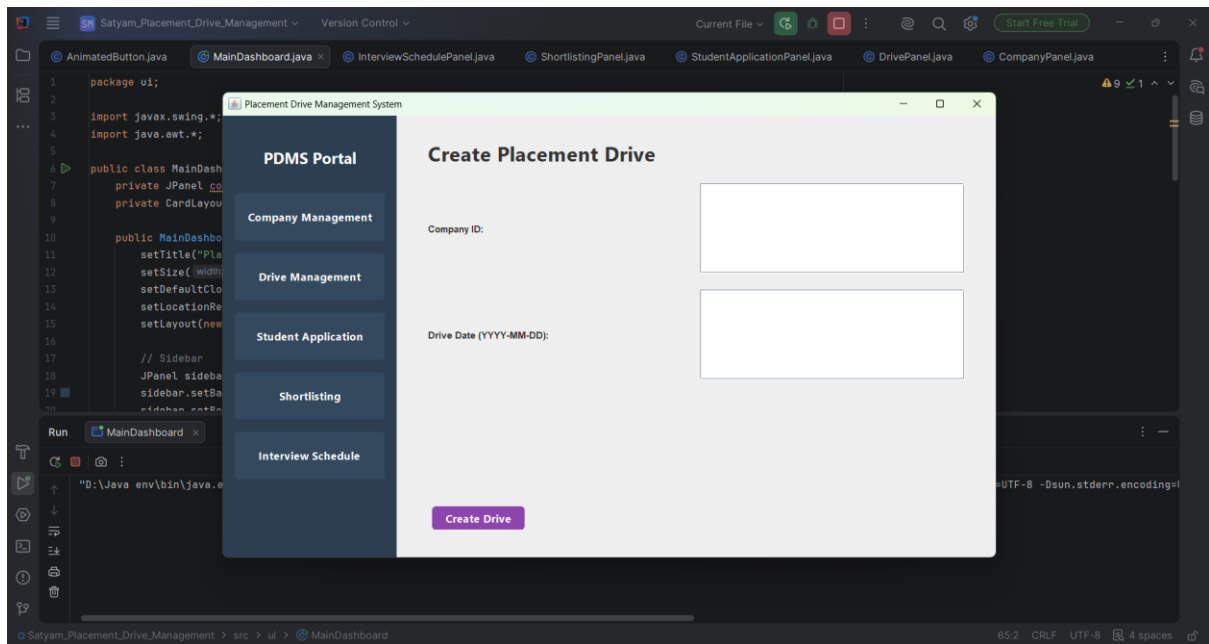
- **Drive Management Screen (DrivePanel.java):**
- **Purpose:** Facilitates Step 2 of the operational flow: "Create Drive".
- **Fields:** Captures Company ID (foreign key linkage) and Drive Date.
- **Action:** Submits drive details to the system, integrating with DriveDAO.addDrive().



```
1 package ui;
2
3 import javax.swing.*;
4 import java.awt.*;
5
6 public class DrivePanel extends JPanel { 1 usage
7     private JTextField txtCompanyId; txtDate; 4 usages
8
9     public DrivePanel() { 1 usage
10         setLayout(new BorderLayout(hgap: 20, vgap: 20));
11         setBorder(BorderFactory.createEmptyBorder(top: 30, left: 40, bottom: 30, right: 40));
12
13         JLabel title = new JLabel(text: "Create Placement Drive");
14         title.setFont(new Font(fontName: "Segoe UI", Font.BOLD, size: 26));
15         add(title, BorderLayout.NORTH);
16
17         JPanel formPanel = new JPanel(new GridLayout(rows: 3, cols: 2, hgap: 10, vgap: 20));
18         formPanel.add(new JLabel(text: "Company ID:"));
19         txtCompanyId = new JTextField();
20         formPanel.add(txtCompanyId);
21
22         formPanel.add(new JLabel(text: "Drive Date (YYYY-MM-DD):"));
23         txtDate = new JTextField();
24         formPanel.add(txtDate);
25
26         AnimatedButton btnSubmit = new AnimatedButton(text: "Create Drive", new Color(r: 142, g: 68, b: 173), new Color(r: 155, g: 89, b: 182));
27         btnSubmit.addActionListener(ActionEvent e -> submit());
28
29         JPanel buttonPanel = new JPanel(new FlowLayout(FlowLayout.LEFT));
30         buttonPanel.add(btnSubmit);
31
32         add(formPanel, BorderLayout.CENTER);
33     }
34 }
```



```
6 public class DrivePanel extends JPanel { 1 usage
7     public DrivePanel() { 1 usage
8         formPanel.add(txtCompanyId);
9
10         formPanel.add(new JLabel(text: "Drive Date (YYYY-MM-DD):"));
11         txtDate = new JTextField();
12         formPanel.add(txtDate);
13
14         AnimatedButton btnSubmit = new AnimatedButton(text: "Create Drive", new Color(r: 142, g: 68, b: 173), new Color(r: 155, g: 89, b: 182));
15         btnSubmit.addActionListener(ActionEvent e -> submit());
16
17         JPanel buttonPanel = new JPanel(new FlowLayout(FlowLayout.LEFT));
18         buttonPanel.add(btnSubmit);
19
20         add(formPanel, BorderLayout.CENTER);
21         add(buttonPanel, BorderLayout.SOUTH);
22     }
23
24     private void submit() { 1 usage
25         if (txtCompanyId.getText().trim().isEmpty() || txtDate.getText().trim().isEmpty()) {
26             JOptionPane.showMessageDialog(parentComponent: this, message: "Company ID and Date are required!", title: "Validation Error", JOptionPane.ERROR_MESSAGE);
27             return;
28         }
29         JOptionPane.showMessageDialog(parentComponent: this, message: "Drive scheduled successfully!", title: "Success", JOptionPane.INFORMATION_MESSAGE);
30         txtCompanyId.setText(""); txtDate.setText("");
31     }
32 }
```



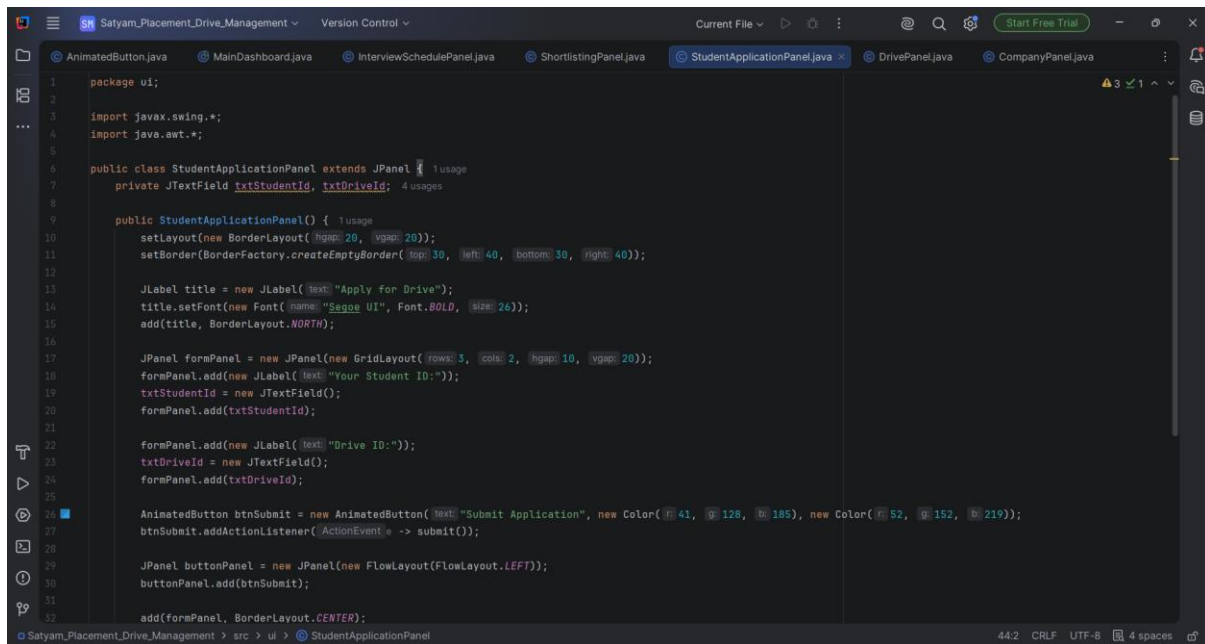
The image displays two screenshots of the 'Create Placement Drive' form within the 'PDMS Portal'.

Top Screenshot: The form is titled 'Create Placement Drive'. It features a sidebar with navigation links: 'PDMS Portal', 'Company Management', 'Drive Management', 'Student Application', 'Shortlisting', and 'Interview Schedule'. The main form area contains two input fields: 'Company ID:' with the value '92634897236' and 'Drive Date (YYYY-MM-DD):' with the value '2026-04-30'. A 'Create Drive' button is located at the bottom left of the form.

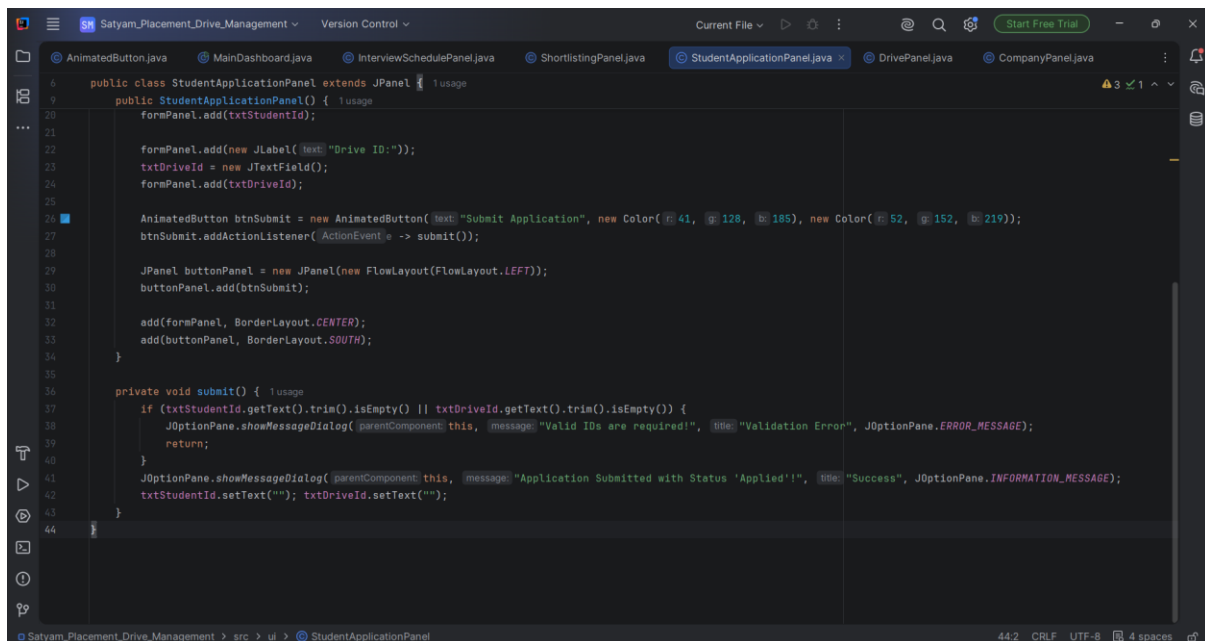
Bottom Screenshot: This screenshot shows the same form, but with a 'Success' dialog box overlaying the 'Drive Date' field. The dialog box contains the message 'Drive scheduled successfully!' and an 'OK' button.

- **Student Application Portal (\$StudentApplicationPanel.java):**
- **Purpose:** Facilitates Step 3 and 4 of the operational flow: "Student Applies" and "Application Stored in Database".
- **Fields:** Captures Student ID and the target Drive ID.

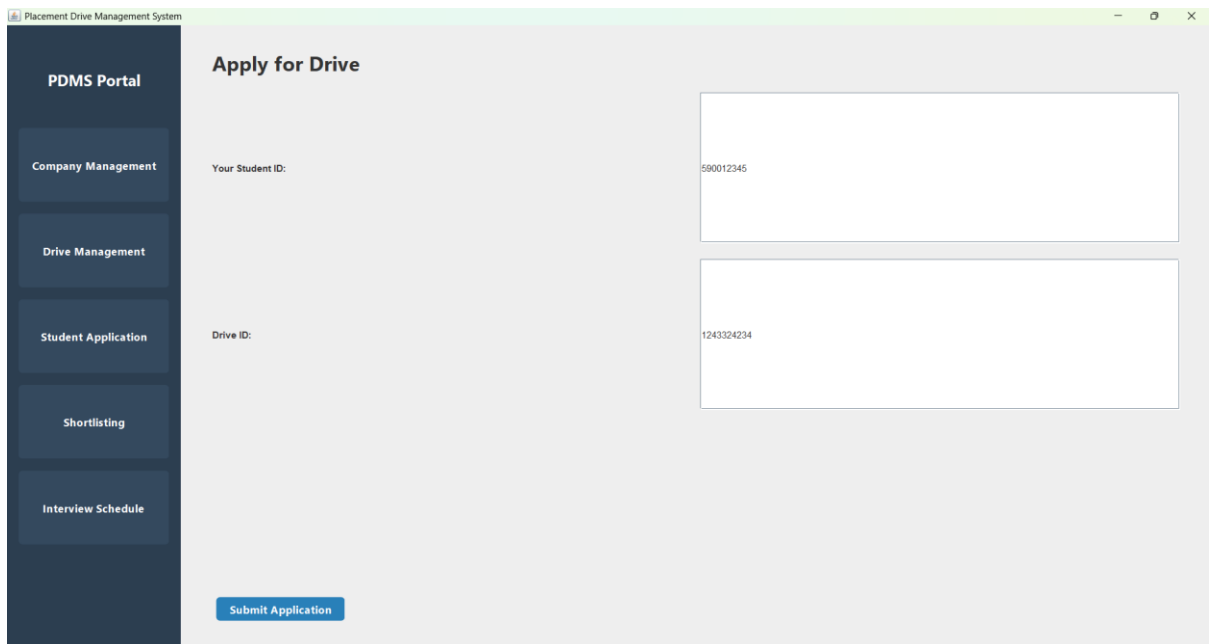
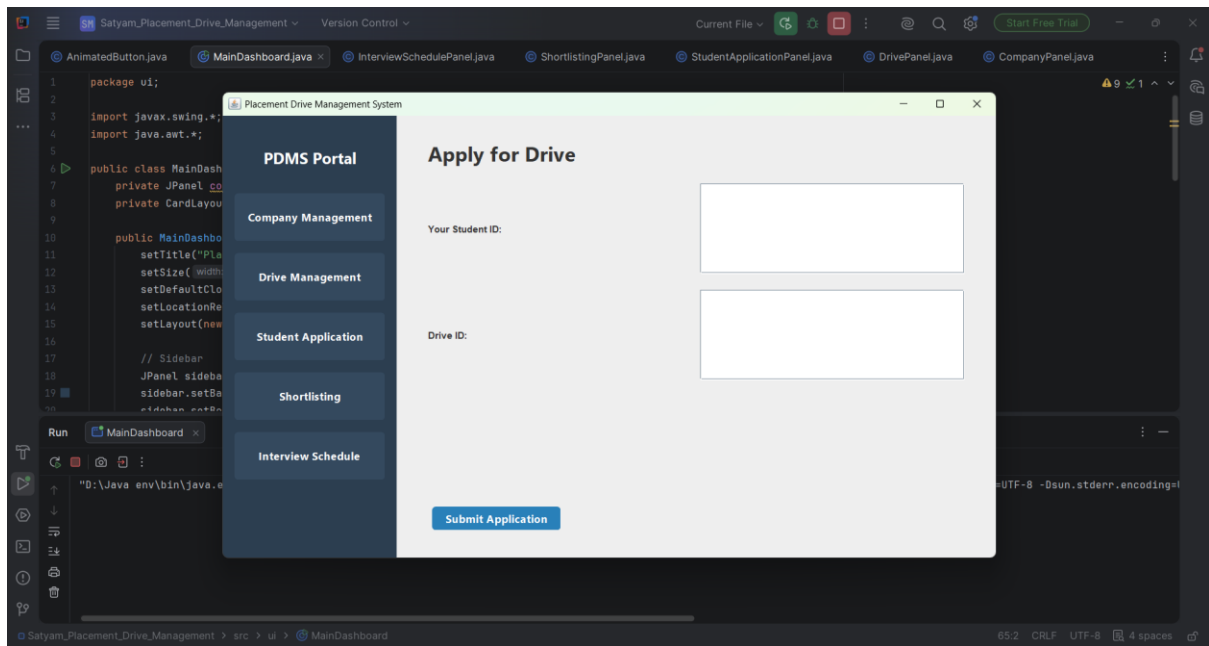
- **Action:** Submits the application with a default status (e.g., "Applied"), mapping to ApplicationDAO.apply().

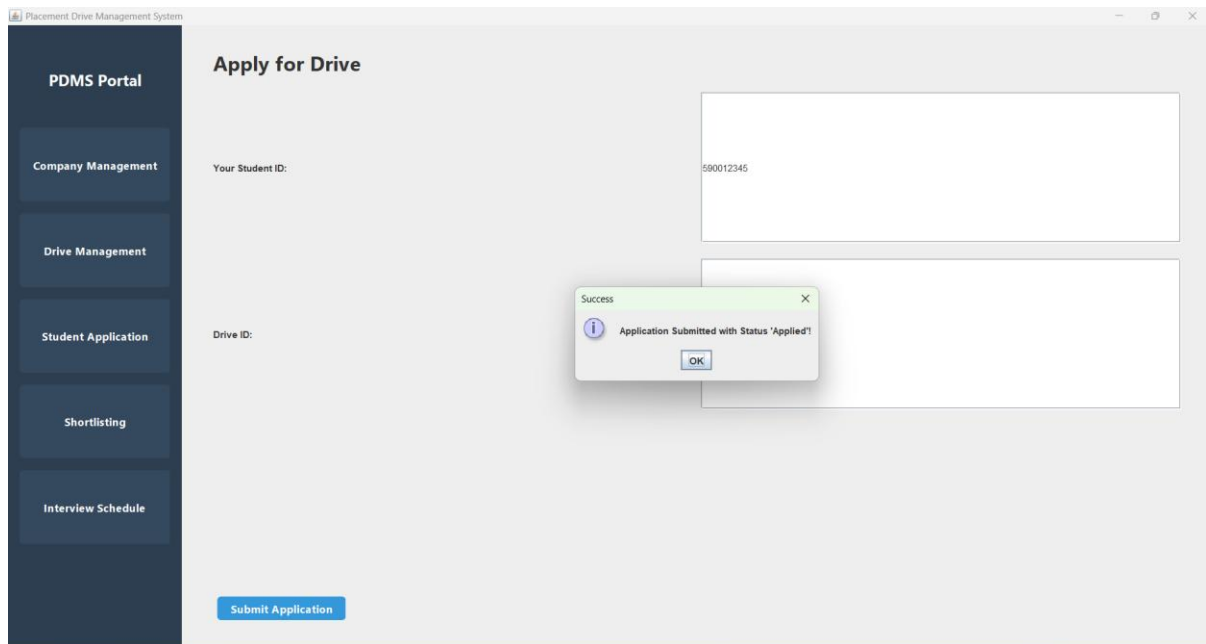


```
1 package ui;
2
3 import javax.swing.*;
4 import java.awt.*;
5
6 public class StudentApplicationPanel extends JPanel { 1 usage
7     private JTextField txtStudentId, txtDriveId; 4 usages
8
9     public StudentApplicationPanel() { 1 usage
10         setLayout(new BorderLayout(10, 10));
11         setBorder(BorderFactory.createEmptyBorder(10, 10, 10, 10));
12
13         JLabel title = new JLabel("Apply for Drive");
14         title.setFont(new Font("Segoe UI", Font.BOLD, 26));
15         add(title, BorderLayout.NORTH);
16
17         JPanel formPanel = new JPanel(new GridLayout(3, 2, 10, 20));
18         formPanel.add(new JLabel("Your Student ID:"));
19         txtStudentId = new JTextField();
20         formPanel.add(txtStudentId);
21
22         formPanel.add(new JLabel("Drive ID:"));
23         txtDriveId = new JTextField();
24         formPanel.add(txtDriveId);
25
26         AnimatedButton btnSubmit = new AnimatedButton("Submit Application", new Color(41, 128, 185), new Color(52, 152, 219));
27         btnSubmit.addActionListener(new ActionListener() {
28             @Override
29             public void actionPerformed(ActionEvent e) {
30                 submit();
31             }
32         });
33
34         JPanel buttonPanel = new JPanel(new FlowLayout(FlowLayout.LEFT));
35         buttonPanel.add(btnSubmit);
36
37         add(formPanel, BorderLayout.CENTER);
38         add(buttonPanel, BorderLayout.SOUTH);
39     }
40
41     private void submit() { 1 usage
42         if (txtStudentId.getText().trim().isEmpty() || txtDriveId.getText().trim().isEmpty()) {
43             JOptionPane.showMessageDialog(parentComponent, "Valid IDs are required!", "Validation Error", JOptionPane.ERROR_MESSAGE);
44             return;
45         }
46         JOptionPane.showMessageDialog(parentComponent, "Application Submitted with Status 'Applied'!", "Success", JOptionPane.INFORMATION_MESSAGE);
47         txtStudentId.setText(""); txtDriveId.setText("");
48     }
49 }
```

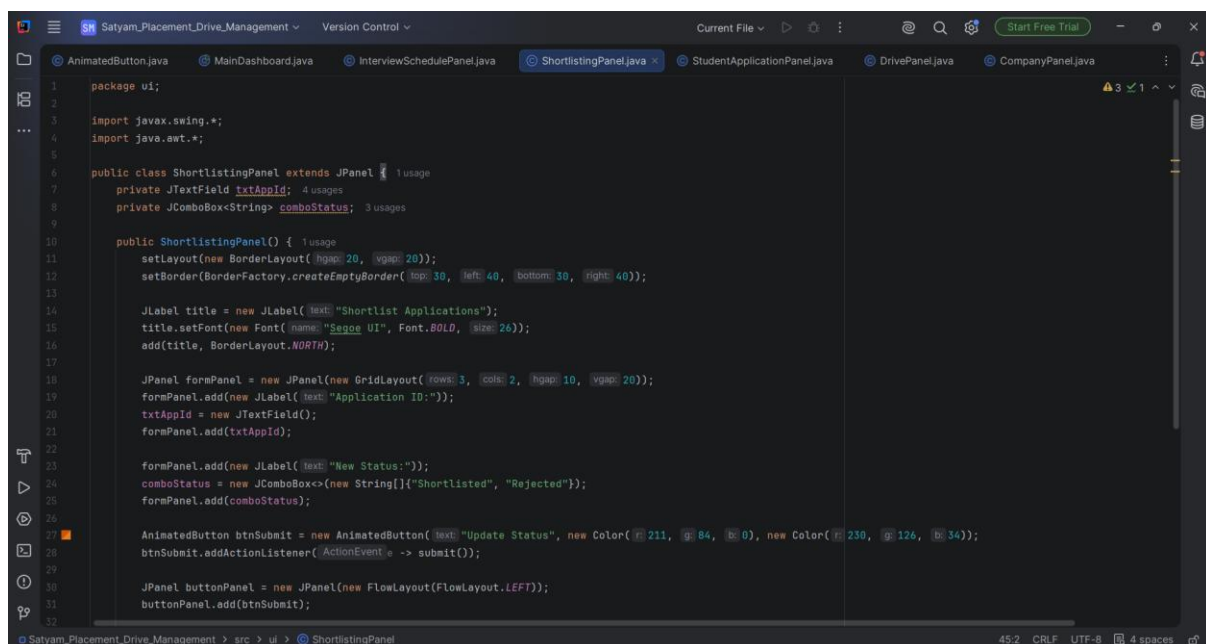


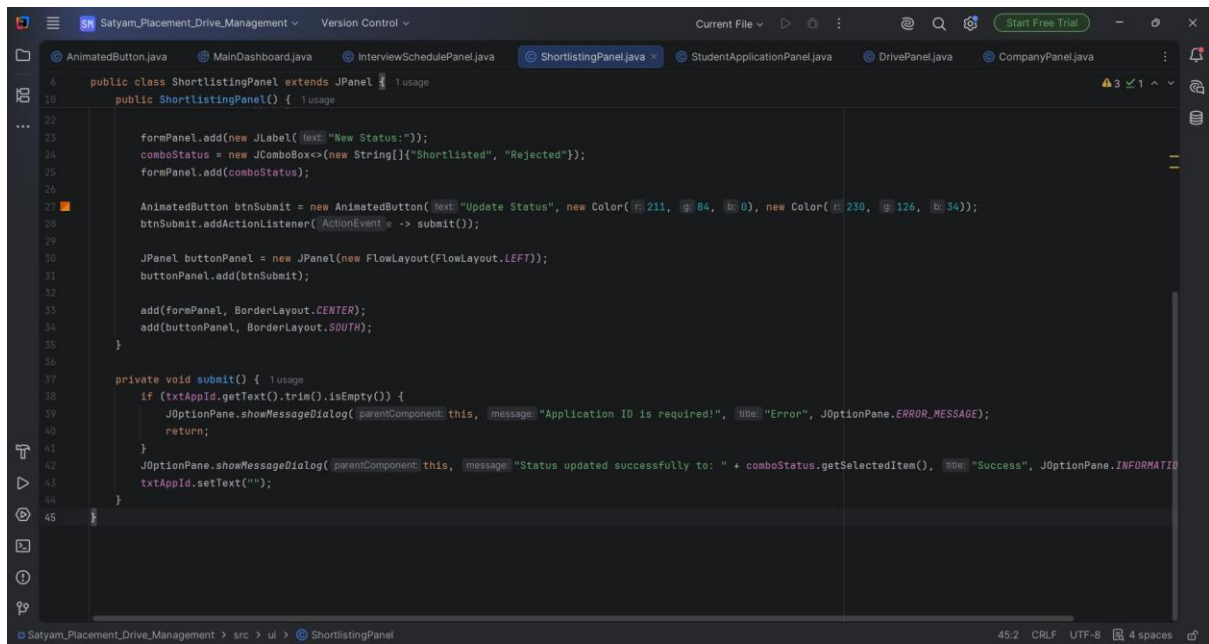
```
6 public class StudentApplicationPanel extends JPanel { 1 usage
9     public StudentApplicationPanel() { 1 usage
20         formPanel.add(txtStudentId);
21
22         formPanel.add(new JLabel("Drive ID:"));
23         txtDriveId = new JTextField();
24         formPanel.add(txtDriveId);
25
26         AnimatedButton btnSubmit = new AnimatedButton("Submit Application", new Color(41, 128, 185), new Color(52, 152, 219));
27         btnSubmit.addActionListener(new ActionListener() {
28             @Override
29             public void actionPerformed(ActionEvent e) {
30                 submit();
31             }
32         });
33
34         JPanel buttonPanel = new JPanel(new FlowLayout(FlowLayout.LEFT));
35         buttonPanel.add(btnSubmit);
36
37         add(formPanel, BorderLayout.CENTER);
38         add(buttonPanel, BorderLayout.SOUTH);
39     }
40
41     private void submit() { 1 usage
42         if (txtStudentId.getText().trim().isEmpty() || txtDriveId.getText().trim().isEmpty()) {
43             JOptionPane.showMessageDialog(parentComponent, "Valid IDs are required!", "Validation Error", JOptionPane.ERROR_MESSAGE);
44             return;
45         }
46         JOptionPane.showMessageDialog(parentComponent, "Application Submitted with Status 'Applied'!", "Success", JOptionPane.INFORMATION_MESSAGE);
47         txtStudentId.setText(""); txtDriveId.setText("");
48     }
49 }
```



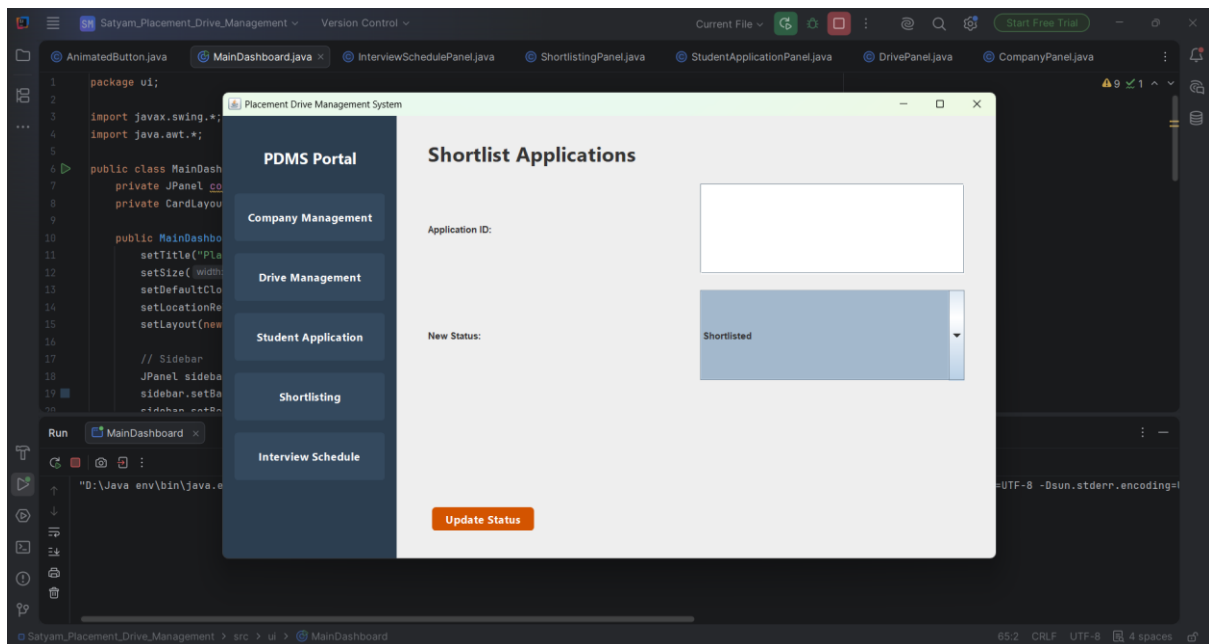


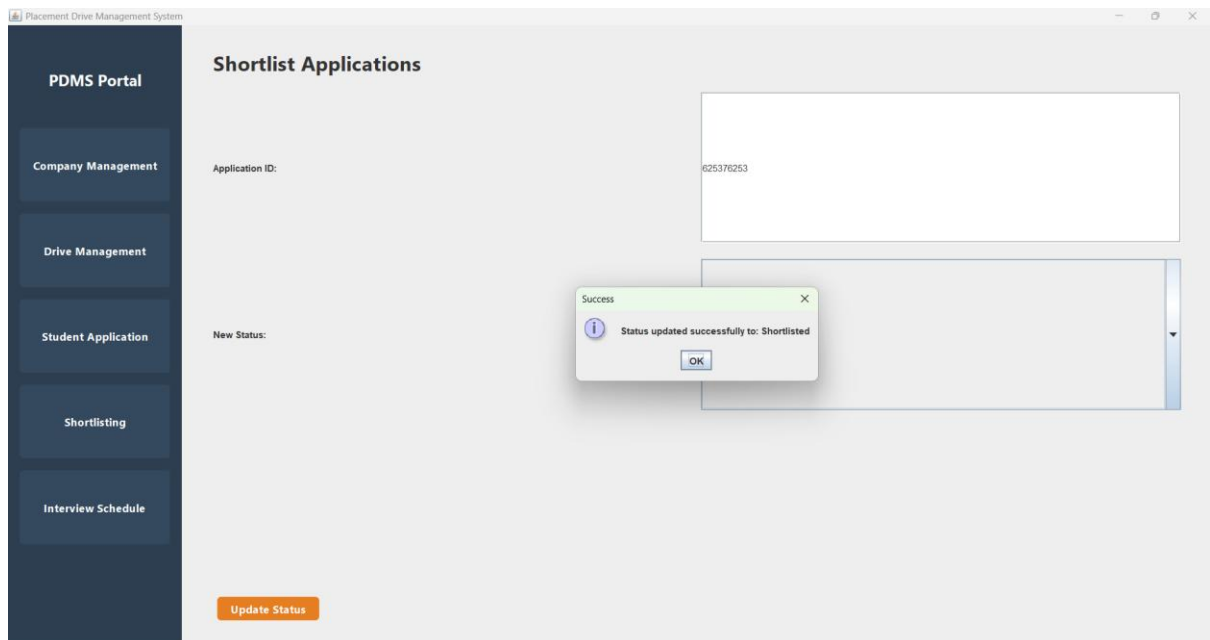
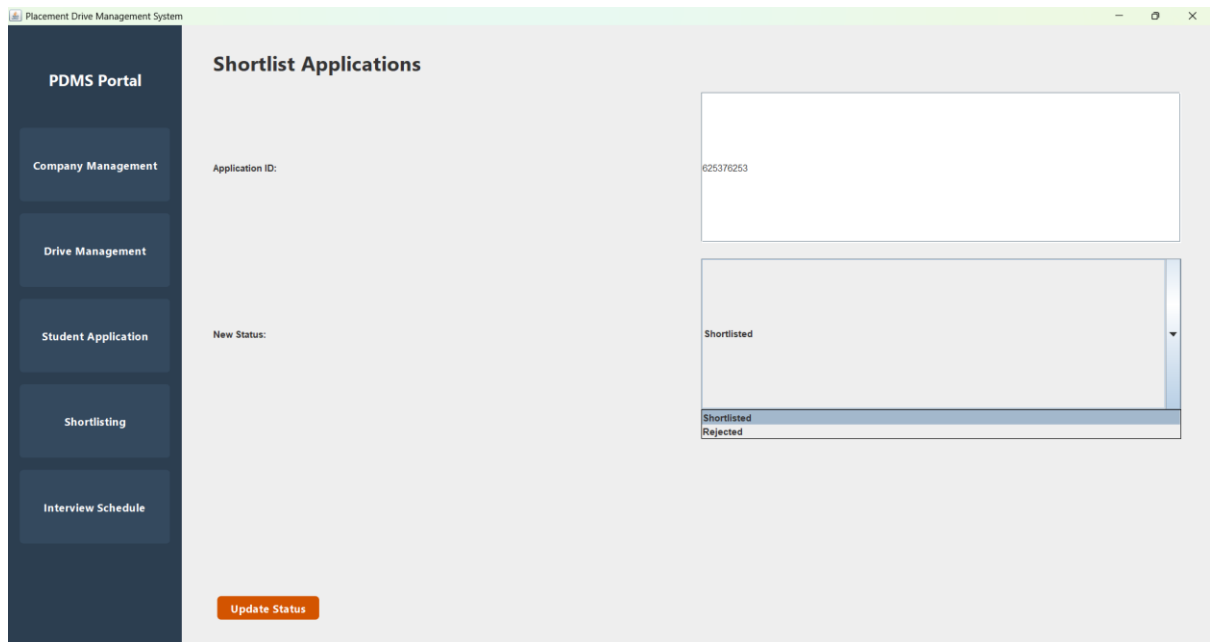
- **Shortlisting Screen (ShortlistingPanel.java):**
- **Purpose:** Allows administrators to evaluate applications.
- **Fields:** Captures the Application ID and uses a dropdown (JComboBox) to select the new status (e.g., "Shortlisted", "Rejected").





```
4 public class ShortlistingPanel extends JPanel { 1 usage
10 public ShortlistingPanel() {
22     formPanel.add(new JLabel(new JLabel("New Status:")));
23     comboStatus = new JComboBox<>(new String[]{"Shortlisted", "Rejected"});
24     formPanel.add(comboStatus);
25
26     AnimatedButton btnSubmit = new AnimatedButton(new JLabel("Update Status"), new Color(0, 211, 84), new Color(0, 230, 126), new Color(0, 84, 211));
27     btnSubmit.addActionListener(new ActionListener() {
28         public void actionPerformed(ActionEvent e) { submit(); }
29     });
30
31     JPanel buttonPanel = new JPanel(new BorderLayout());
32     buttonPanel.add(btnSubmit);
33
34     add(formPanel, BorderLayout.CENTER);
35     add(buttonPanel, BorderLayout.SOUTH);
36 }
37
38 private void submit() { 1 usage
39     if (txtAppId.getText().trim().isEmpty()) {
40         JOptionPane.showMessageDialog(parentComponent, this, "Application ID is required!", "Error", JOptionPane.ERROR_MESSAGE);
41         return;
42     }
43     JOptionPane.showMessageDialog(parentComponent, this, "Status updated successfully to: " + comboStatus.getSelectedItem(), "Success", JOptionPane.INFORMATION_MESSAGE);
44     txtAppId.setText("");
45 }
```





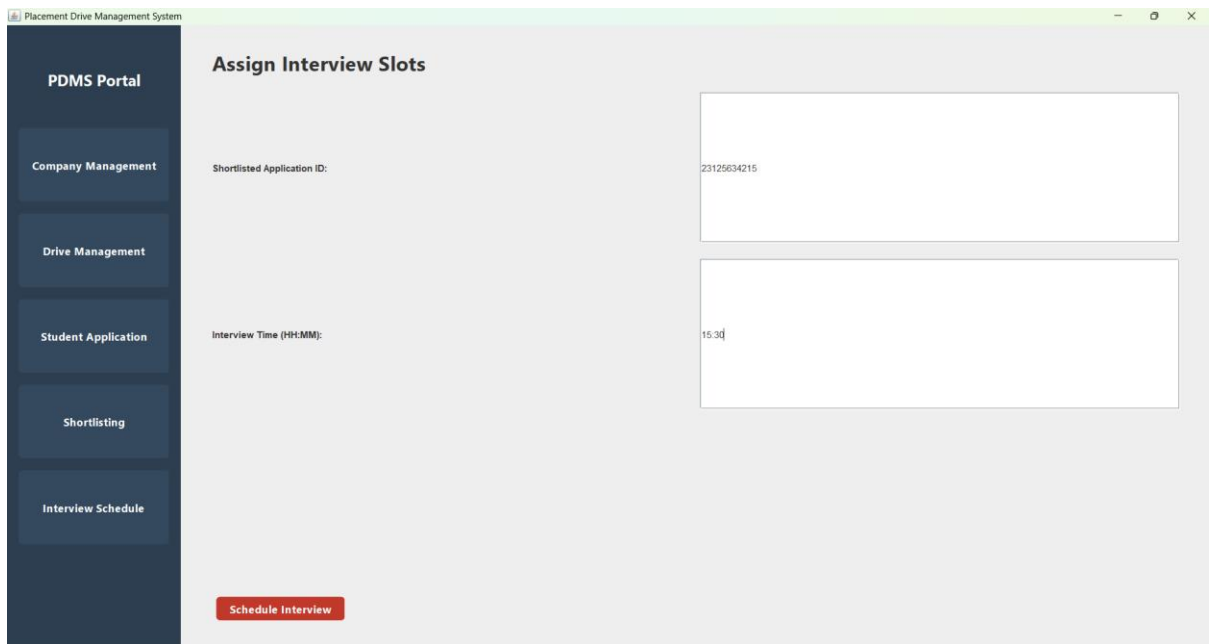
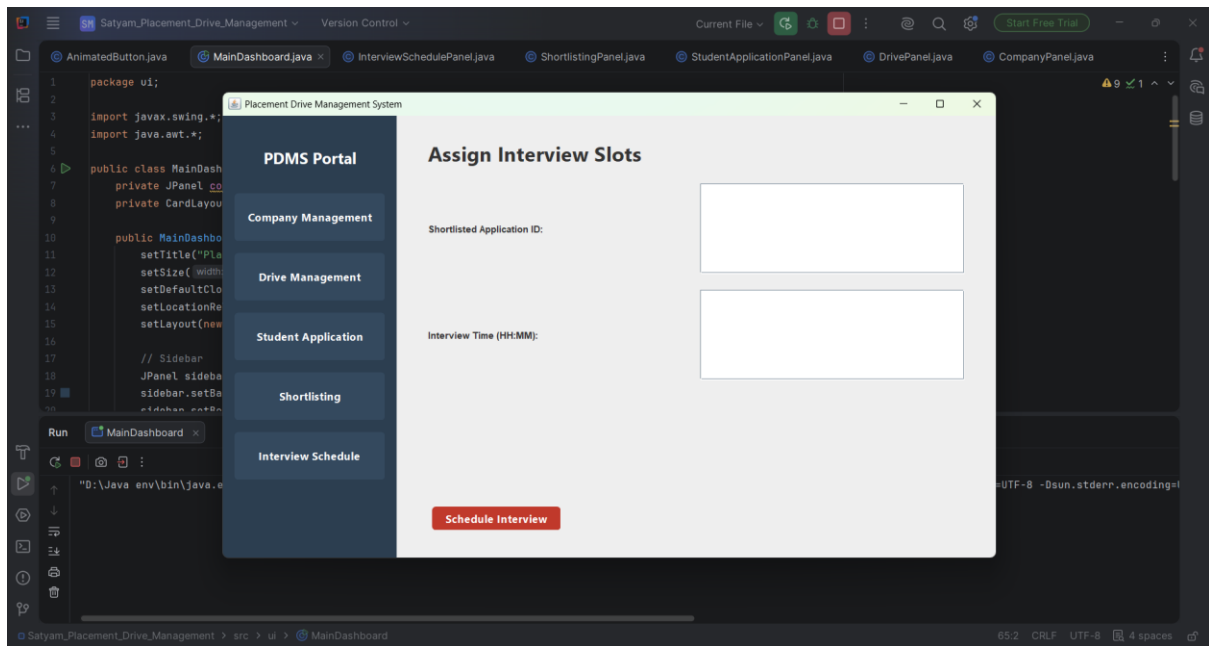
- **Interview Schedule Screen (InterviewSchedulePanel.java):**
- **Purpose:** The final step of the recruitment workflow.
- **Fields:** Maps a shortlisted Application ID to an Interview Time.

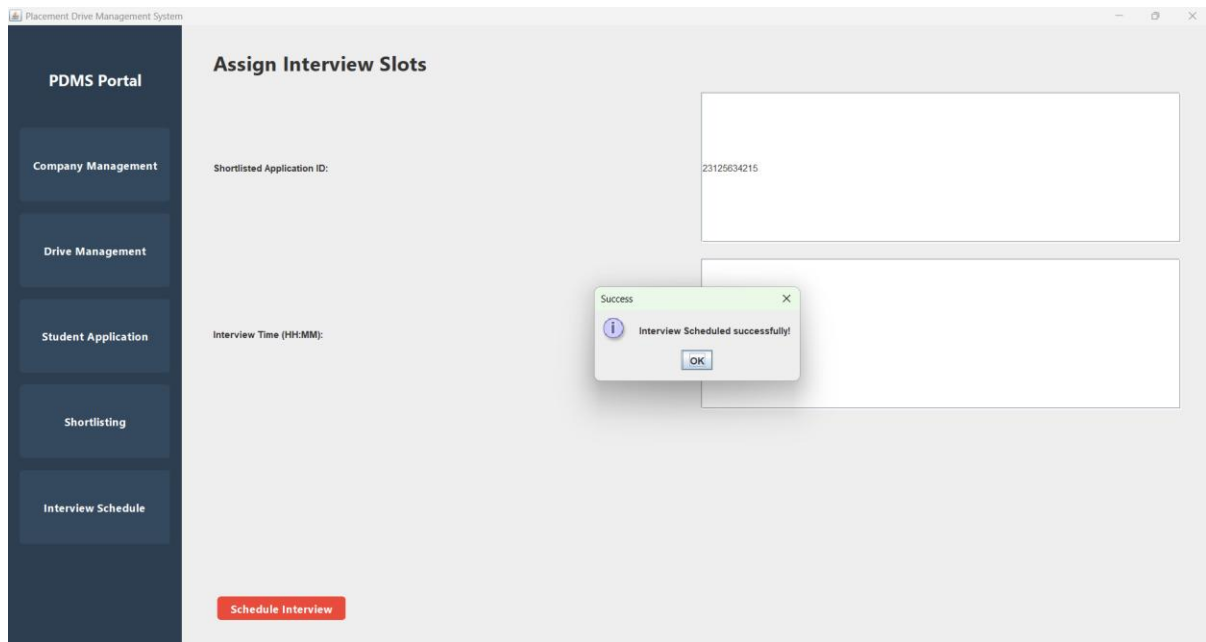
```
Satyam_Placement_Drive_Management Version Control
AnimatedButton.java MainDashboard.java InterviewSchedulePanel.java ShortlistingPanel.java StudentApplicationPanel.java DrivePanel.java CompanyPanel.java

1 package ui;
2
3 import javax.swing.*;
4 import java.awt.*;
5
6 public class InterviewSchedulePanel extends JPanel { 1 usage
7     private JTextField txtAppId, txtTime; 4 usages
8
9     public InterviewSchedulePanel() { 1 usage
10         setLayout(new BorderLayout(10, 10));
11         setBorder(BorderFactory.createEmptyBorder(10, 10, 10, 10));
12
13         JLabel title = new JLabel("Assign Interview Slots");
14         title.setFont(new Font("Segoe UI", Font.BOLD, 26));
15         add(title, BorderLayout.NORTH);
16
17         JPanel formPanel = new JPanel(new GridLayout(2, 1, 10, 10));
18         formPanel.add(new JLabel("Shortlisted Application ID:"));
19         txtAppId = new JTextField();
20         formPanel.add(txtAppId);
21
22         formPanel.add(new JLabel("Interview Time (HH:MM):"));
23         txtTime = new JTextField();
24         formPanel.add(txtTime);
25
26         AnimatedButton btnSubmit = new AnimatedButton("Schedule Interview", new Color(192, 57, 43), new Color(231, 76, 60));
27         btnSubmit.addActionListener(e -> submit());
28
29         JPanel buttonPanel = new JPanel(new FlowLayout(FlowLayout.LEFT));
30         buttonPanel.add(btnSubmit);
31
32         add(formPanel, BorderLayout.CENTER);
33     }
34 }
```

```
Satyam_Placement_Drive_Management Version Control
AnimatedButton.java MainDashboard.java InterviewSchedulePanel.java ShortlistingPanel.java StudentApplicationPanel.java DrivePanel.java CompanyPanel.java

6 public class InterviewSchedulePanel extends JPanel { 1 usage
9     public InterviewSchedulePanel() { 1 usage
20         formPanel.add(txtAppId);
21
22         formPanel.add(new JLabel("Interview Time (HH:MM):"));
23         txtTime = new JTextField();
24         formPanel.add(txtTime);
25
26         AnimatedButton btnSubmit = new AnimatedButton("Schedule Interview", new Color(192, 57, 43), new Color(231, 76, 60));
27         btnSubmit.addActionListener(e -> submit());
28
29         JPanel buttonPanel = new JPanel(new FlowLayout(FlowLayout.LEFT));
30         buttonPanel.add(btnSubmit);
31
32         add(formPanel, BorderLayout.CENTER);
33         add(buttonPanel, BorderLayout.SOUTH);
34     }
35
36     private void submit() { 1 usage
37         if (txtAppId.getText().trim().isEmpty() || txtTime.getText().trim().isEmpty()) {
38             JOptionPane.showMessageDialog(parentComponent, this, "Application ID and Time are required!", "Error", JOptionPane.ERROR_MESSAGE);
39             return;
40         }
41         JOptionPane.showMessageDialog(parentComponent, this, "Interview Scheduled successfully!", "Success", JOptionPane.INFORMATION_MESSAGE);
42         txtAppId.setText(""); txtTime.setText("");
43     }
44 }
```





- **Integration Strategy and Execution Order**
- **To avoid the common mistake of building many disconnected screens before completing a single workflow , the integration was executed in the following structured order:**
- **Navigation First:** Built the MainDashboard to establish simple navigation between major screens.
- **UI to Service:** Connected the form submission actions directly to mapped service layer methods.
- **Service to DAO:** Ensured service methods correctly invoked the DAO operations engineered in Phase 3.
- **User Feedback:** Programmed clear JOptionPane success messages (e.g., "Company added successfully") and specific error messages. Generic error popups with no useful feedback were strictly avoided.

- **Database Verification:** Verified that the complete flow (e.g., User clicks "Submit Application" → DAO executes INSERT INTO applications) correctly reflected in the MySQL backend.
- **Input Validation Strategy**
- **A major focus of this phase was data integrity. The application visibly validates data before attempting any database commits.**
- **Empty Field Checks:** Forms are scanned for empty fields (e.g., trim().isEmpty()). If critical data like Company ID or Student ID is missing, the action is blocked, and the user receives a targeted warning message.
- **Data Type Checks:** The UI ensures that numerical ID fields are not submitted with alphabetical characters, catching format exceptions before they hit the DAO layer.
- **Project Folder Structure**
- **Building upon the Phase 3 structure, the project directory has been expanded to accommodate the clean architectural layers:**

```
PlacementSystem/
├── lib/
│   └── mysql-connector-j-9.6.0.jar
├── src/
│   ├── db/
│   │   └── DBConnection.java
│   ├── dao/
│   │   ├── CompanyDAO.java
│   │   ├── DriveDAO.java
│   │   └── ApplicationDAO.java
│   ├── model/
│   │   ├── Company.java
│   │   ├── Drive.java
│   │   └── Application.java
│   ├── service/                                <-- Added in Phase 4
│   │   ├── DriveService.java
│   │   └── ApplicationService.java
│   ├── ui/                                    <-- Added in Phase 4
│   │   ├── DashboardFrame.java
│   │   ├── CompanyPanel.java
│   │   └── ApplicationPanel.java
│   └── Main.java
└── database.sql
```

- **Design Rules Adherence**
- Strict separation of concerns was maintained.
- **No SQL queries were placed inside UI event handlers.**
All database logic remains firmly in the DAO layer, bridging through the Service layer.
- **Common Mistakes Avoided**
- During development, the team actively avoided the following pitfalls outlined in the phase guidelines:

- We avoided building too many screens simultaneously; instead, we ensured one complete workflow was finished before moving on.
 - We established a clear navigation flow rather than disjointed, disconnected windows.
 - We strictly prohibited directly calling SQL statements from UI event handlers.
 - We replaced generic error popups with highly specific, useful feedback messages.
-
- **Phase 4 Milestone Review Checklist**
 - **The integration and UI development have successfully met the phase criteria:**
 - **Workflow Integrity:** One complete workflow (Add Company → Create Drive → Student Applies) runs seamlessly from the UI down to the DB and back.
 - **Layer Separation:** Screen actions are exclusively connected to service logic; direct SQL calls from UI event handlers have been completely avoided.
 - **Data Security:** Invalid inputs are properly blocked with visible, descriptive feedback.
 - **Usability:** With the inclusion of interactive animations and a unified dashboard, the project is highly usable and ready for the milestone review.